

Content- and Session-Aware Anomaly Detection for Web Server Logs Using a Fused Autoencoder and Variational LSTM Framework: A Cross-Dataset Evaluation

Abdul Rehman *, Raja Ujjan and Muhsin Hassanu *

Department of Computing and Cybersecurity, University of the West of Scotland, London Campus, London E14 2BA, UK

* Correspondence: abdulrehmankhokhar25@gmail.com (A.R.); muhsin.hassanu@uws.ac.uk (M.H.)

Abstract

Web server logs are rich in security-relevant information, but rule-based inspection misses unseen attacks, and most anomaly-detection studies validate on a single dataset, risking overstated generalisation. This paper presents a hybrid framework fusing a content-based autoencoder that scores individual HTTP requests with a session-based variational LSTM autoencoder that scores genuine per-client request sequences, evaluated across eight datasets spanning three log formats, including the public CSIC 2010 HTTP dataset, under within- and cross-dataset threshold-transfer protocols. Fixing a session-construction fault that mixed different clients' requests raises the session branch's AUC on CSIC 2010 from 0.63 to 0.97, though it scores nothing for clients with only one or two requests, so its benefit depends on real session structure. Fusion is not uniformly beneficial: on CSIC 2010, weighting fully toward the session branch beats every blended weight tested. A threshold calibrated once on normal CSIC 2010 traffic and transferred unchanged to three attack-heavy datasets raises fused F1 five- to six-fold over a per-dataset threshold, since those datasets are too attack-heavy for self-calibration to be meaningful. The framework also achieves a lower false positive rate than a labelled Random Forest baseline at comparable or better AUC than unsupervised baselines, and runs fast enough for near-real-time use. Session structure and threshold calibration can matter as much as model choice.

Keywords: web server logs; anomaly detection; autoencoder; variational LSTM; session modelling; score fusion; cross-dataset generalisation

1. Introduction

Web applications underpin most critical digital services in use today, from banking and healthcare to e-commerce and cloud infrastructure, and their continuous, public-facing exposure makes them a persistent target for attackers [1]. Every request a web server processes is recorded in an access log, together with metadata such as the request method, path, response status, and client identifiers. These logs are one of the richest available sources of security-relevant information: in principle, they capture both the legitimate use of an application and the traces left by attackers probing or exploiting it. In practice, however, the volume and variety of this data make manual inspection infeasible for any application of meaningful scale, and organisations have long relied on signature-based intrusion detection and rule-based filtering to triage it [1].

Signature- and rule-based approaches are effective against known attack patterns but are structurally unable to detect zero-day or previously unseen attacks, since they can

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Future Internet* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

only recognise what has already been catalogued. This has motivated a shift towards anomaly detection, which instead models what *normal* traffic looks like and flags deviations from it, without requiring the deviation to match a predefined signature [2]. Anomaly detection is a natural fit for web security because it does not depend on a labelled corpus of known attacks – which is difficult to obtain, quickly goes stale, and can never be assumed complete – but it introduces its own difficulties. Web traffic is highly non-stationary: normal behaviour drifts as applications change, and it can vary enormously between different endpoints, user populations, and even times of day. Attack traffic is also typically a tiny fraction of the total volume, so any evaluation of an anomaly detector must be designed around severe class imbalance rather than raw accuracy [2].

Deep learning has extended anomaly detection to log and web-traffic data by learning representations directly from raw or lightly processed records, rather than requiring hand-engineered features. Sequence models such as LSTMs [3] have been used to learn the normal structure of a stream of log events and flag deviations from it [4], and DeepLog demonstrated that treating a system log as a natural-language-like sequence allows an LSTM to learn normal execution patterns and detect anomalies as departures from them [5]. Applied to HTTP traffic specifically, this behavioural, sequence-oriented view is complementary to a second, request-level view: individual HTTP requests can themselves be modelled directly, using an autoencoder trained to reconstruct the structural and payload characteristics of normal requests, so that unusually large reconstruction error flags anomalous content such as injection payloads or malformed parameters. Recent work combining a variational LSTM autoencoder [6,7] with a deviation network for exactly this purpose has shown that this class of model is well suited to detecting web attacks directly from HTTP weblogs [8].

These two views, however, are not interchangeable. A content-based model that scores individual requests in isolation cannot see attacks that only become apparent across a sequence of otherwise unremarkable requests – reconnaissance-style scanning, credential stuffing, or low-and-slow probing, for example, where each individual request may be syntactically valid. A behaviour-based model that scores sequences of requests grouped by client can catch exactly this class of attack, but it depends on there being a genuine sequence to model in the first place: if a client only issues one or two requests, or if requests from a given client cannot be reliably grouped together, there is no behavioural signal to learn from. This dependency on session structure is not merely a theoretical caveat – as this study’s own experiments later show (Section 5), it materially determines whether a session-based branch can contribute anything at all for a given traffic sample.

Because content-based and behaviour-based detectors fail in different, largely non-overlapping ways, combining their outputs is a natural way to cover more of the attack surface than either can alone. Hybrid, fusion-based frameworks that integrate multiple detection signals have been shown to improve robustness and stability relative to single-model detectors, particularly under the kind of severe class imbalance and limited labelled data that characterises real security settings [9]. Realising this benefit in practice, however, depends heavily on how the resulting system is evaluated. A large share of the anomaly-detection literature reports results on a single dataset, which risks overstating how well a method will generalise once deployed against traffic with different structure, format, or attack composition than whatever it was tuned against. This concern is well established in the wider intrusion-detection literature, where the design of benchmark datasets themselves – their realism, diversity, and reproducibility – has long been recognised as a determining factor in whether reported results are meaningful outside the dataset they were produced on [10].

Despite the individual maturity of content-based autoencoders, behaviour-based sequence models, and fusion-based detection frameworks, their combined application to web server logs specifically, evaluated across genuinely different log formats and under a strict cross-dataset protocol – where a detection threshold is fixed on one data source and applied unchanged to others, rather than recalibrated on each dataset in turn – remains underexplored. Many studies that do combine content and behavioural signals validate the result on a single log format and a single traffic sample, leaving open how such a system behaves when the underlying session structure, log schema, or class balance of the target traffic differs substantially from what the model was built on.

To address this gap, this work presents a hybrid anomaly detection framework for web server logs that combines a content-based autoencoder, trained to reconstruct the structural features of individual HTTP requests, with a session-based variational LSTM autoencoder, trained on genuine per-client request sequences rather than an arbitrary sliding window over the raw log stream. The outputs of the two branches are combined through score-level fusion, and the resulting framework is evaluated not only within a single dataset but across three structurally different sources: two self-generated Apache-style access-log corpora and a structured Nginx JSON log, together with the publicly available CSIC 2010 HTTP dataset, which additionally serves as the source domain for a cross-dataset threshold-transfer evaluation in which a single fixed decision threshold, calibrated once on genuinely benign traffic, is applied unchanged to unseen target datasets.

The remainder of this paper is organised as follows. Section 2 sets out the specific contributions of this work. Section 3 reviews related work on anomaly detection for web and log data, content- and behaviour-based deep learning models, and hybrid / fusion-based detection frameworks. Section 4 describes the datasets, preprocessing, feature representation, model architectures, and evaluation protocol. Section 5 presents the experimental results, including the branch ablation, fusion-weight sensitivity, classical-baseline comparison, and cross-dataset threshold-transfer experiments. Section 6 discusses these findings and their practical implications, and Section 7 concludes the paper and outlines directions for future work.

2. Research Contributions

This paper makes the following contributions.

First, we build a hybrid system that finds anomalies in web server logs using two parts. One part checks each request on its own, using an autoencoder. The other part checks a sequence of requests from the same client, using a variational LSTM. We combine the two scores into one final score, so the system can catch both strange single requests and strange patterns of behaviour over time.

Second, we find and fix a real problem in how the behaviour part was trained. In the old version, the model was trained using a plain sliding window over the whole log. This window did not check which client sent each request, so requests from different clients could end up mixed in the same window. We fix this by grouping requests by client (IP address and user agent) first, so each window only holds requests from one real client. This fix makes a big difference: on the CSIC 2010 dataset, where one client sends many requests in a row, the behaviour part's AUC score rises from about 0.63 to about 0.97 after the fix.

Third, this work reports an honest and useful finding about the data itself. Some of our datasets have only one request per client. In real per-session detection, if a client only sends one request, there is no sequence to check, so the behaviour part gives no score at all for that data. We treat this as a fact about the data, not a failure of the method, and we explain it clearly instead of hiding it. This shows that a true session-based model only helps when the traffic actually has multi-request sessions in it.

Fourth, we test the system on more than one kind of log. We use two Apache-style logs we built ourselves, one Nginx JSON log, and the public CSIC 2010 HTTP dataset. Testing across different log formats and different datasets shows whether the method still works when the data changes shape.

Fifth, we run a cross-dataset test that checks if a threshold set on one dataset still works on another. We set the detection threshold using only the CSIC data, and only its normal, non-attack traffic. Then we use that same threshold, with no change, on three other datasets. The result is surprising: F1 score goes up, not down, when we use the frozen CSIC threshold instead of a threshold tuned on each dataset on its own. For example, on the Access Eval Mix 2000 dataset, the fused F1 score rises from about 0.12 (tuned on that dataset) to about 0.67 (using the frozen CSIC threshold). This happens because these test datasets contain a lot of attack traffic, so a threshold tuned only on them is not a fair "normal traffic" threshold. This is a useful, real finding about how to set thresholds in practice.

Sixth, we compare our system with simple baseline methods: Random Forest, which uses attack labels during training, and Isolation Forest and One-Class SVM, which do not use labels, like our own method. We also measure how fast the system runs. On average, scoring one request takes about 0.14 ms for the content part and about 0.08 ms for the behaviour part when done one request at a time, and scoring in batches is much faster, at about 630,000 lines per second. This shows the system could run in near real time.

3. Related Work

3.1. Anomaly Detection in Web Server Logs

Web server logs record every request a server handles. Each line usually has the request method, the URL, the response code, the time, and the client's IP address. Security teams can use these logs to find attacks, but there are far too many lines to check by hand. For a long time, most systems used rules or signatures to spot bad requests [1]. Rules like this work well for known attacks, but they cannot catch new or changed attacks, because they only look for patterns someone has already written down. This is why anomaly detection is useful: instead of matching known attack patterns, it learns what normal traffic looks like and flags anything that looks different [2]. This can catch attacks that have never been seen before, but it is harder to build well, because normal web traffic changes over time and attacks are a very small part of all traffic.

3.2. Classical Machine Learning Methods

Before deep learning became common, many studies used classic machine learning methods for this problem. Isolation Forest is one of the most used. It works by randomly splitting the data again and again; points that get cut off quickly, after only a few splits, are treated as anomalies [11]. This method does not need labelled attack data, runs fast, and is simple to set up, which is why it is still used today. For example, one recent study applied Isolation Forest directly to web traffic logs from an e-commerce site and showed it can tell normal and anomalous requests apart without needing attack labels [12]. One-Class SVM is another classic unsupervised method, learning a boundary around normal data in feature space and flagging anything that falls outside it [13]; like Isolation Forest, it needs no attack labels, which is why both are used as unsupervised baselines later in this paper (Section 5). Where attack labels are available, Random Forest [14], an ensemble of decision trees, remains a strong and widely used supervised baseline and is included here for the same reason. All three of these classical methods, however, typically need features to be picked and built by hand, and they can struggle when the traffic pattern changes.

3.3. Content-Based Deep Learning

Deep learning removes the need for hand-built features by learning directly from the raw or lightly processed request data. One common approach is the autoencoder. An autoencoder learns to compress a normal request into a small representation and then rebuild it. If the model is trained only on normal requests, it becomes good at rebuilding normal requests but bad at rebuilding attack requests, so a request with a high rebuild error can be flagged as an anomaly; this reconstruction-based view of anomaly scoring, including the variational form used for the session branch in this paper, has been formalised directly as a reconstruction-probability anomaly score [7]. One study used this idea end to end, training a deep autoencoder directly on HTTP requests, and showed it could detect attacks such as SQL injection and cross-site scripting without needing much labelled data [15]. More recent work has compared convolutional and Transformer-based classifiers for recognising malicious intent directly from HTTP requests [16], and one recent study combined character-level LSTM and CNN-BiLSTM models to detect web attacks on the CSIC 2010 dataset together with a second, independently collected dataset, though without a shared decision threshold transferred between them [17]. Models like this are good at finding strange or broken single requests, but they look at each request on its own, so they cannot see attacks that only show up across many requests over time.

3.4. Behaviour-Based Deep Learning

To catch attacks that unfold over several requests, other work models the order and pattern of requests instead of looking at single requests. LSTM networks are built for this, since they can learn from a sequence of past events. DeepLog treats a stream of log lines like a sentence, learning the normal order of events and flagging any sequence that breaks the learned pattern [5]. Later work used a self-attention model, instead of or alongside an LSTM, to learn which parts of a log sequence matter most for telling normal and abnormal apart [4]. Extending this self-attention idea further, a Transformer encoder trained with BERT-style self-supervised objectives has also been used to learn normal log-sequence patterns directly, flagging sequences that deviate from them [18]. Closer to our own work, one study built a variational LSTM autoencoder combined with a deviation network, trained directly on HTTP weblogs, and showed it can detect web attacks by modelling normal session behaviour [8]. Behaviour-based models like these can catch attacks that content-based models miss, such as scanning or repeated probing, but they need enough real, connected requests from the same client to learn from. If a client only sends one or two requests, there is no real sequence to learn from – a point this paper returns to directly in Section 5.

3.5. Hybrid and Fusion-Based Detection

Since content-based and behaviour-based models catch different kinds of attacks, several studies combine both into one system. The idea is to score each request or session with more than one model, then merge the scores into a single final decision. This kind of hybrid, fusion-based design has been shown to work better than either model alone, especially when labelled attack data is scarce and normal traffic looks very different from attack traffic [9]. Building a good fusion system is not simple, though: the two branches usually need to be trained and set up carefully so that one does not drown out the other, and the merged score still needs its own threshold.

3.6. Cross-Dataset Evaluation

Most of the studies above are tested on a single dataset. This is a real problem, because a model that works well on the data it was tuned on may not work as well on new data with

a different shape, format, or mix of attacks. Building good benchmark datasets for intrusion detection, and testing across more than one of them, has long been seen as important for getting results that mean something outside the exact dataset used [10]. A recent study looked at this problem directly for network intrusion detection: models that scored close to perfect when trained and tested on the same dataset dropped close to random guessing when tested on a different dataset, showing how easy it is to overstate a model’s real performance with single-dataset testing [19]. This is the exact gap this paper addresses for web server logs: testing across more than one log format, and running a true cross-dataset threshold test, where the decision threshold is set once and never changed for new data.

3.7. Summary

Table 1 lists the studies discussed above, the kind of method each one uses, the kind of data it was tested on, and the main gap that this paper tries to close.

Table 1. Summary of related studies on web and log anomaly detection.

Study	Method	Data Type	Evaluation	Main Limitation
Scarfone and Mell (2007) [1]	Rule-based IDS /IPS guidance	Network traffic	Guidance document	Rules only catch known attack patterns
Chandola et al. (2009) [2]	Survey	General	Review study	No single detector proposed
Liu et al. (2008) [11]	Isolation Forest	General tabular data	Benchmark datasets	Not evaluated on web request logs
Chua et al. (2024) [12]	Isolation Forest	Web traffic logs	Single dataset	Single detection view, no behaviour model
Schölkopf et al. (2001) [13]	One-Class SVM	General /high-dim. data	Method paper	Not evaluated on web request logs
Breiman (2001) [14]	Random Forest	General tabular data	Benchmark datasets	Supervised; needs attack labels, not web-log specific
Pan et al. (2019) [15]	Stacked denoising autoencoder	HTTP requests	Single dataset	Scores single requests only, no session view
An and Cho (2015) [7]	VAE reconstruction probability	General /tabular data	Benchmark datasets	Not evaluated on web or session data
Tiwari et al. (2024) [16]	CNN vs. Transformer classifier	HTTP requests	Single dataset	Content-only, no session or fusion view
Zhou et al. (2025) [17]	LSTM/CNN-BiLSTM ensemble	HTTP requests	Two datasets	No session view, no threshold-transfer test
Du et al. (2017) [5]	LSTM (DeepLog)	System logs	Single dataset	Not web-request specific
Nedelkoski et al. (2020) [4]	Self-attention classifier	Unstructured logs	Single dataset	No cross-dataset test reported
Guo et al. (2021) [18]	Transformer (LogBERT)	System logs	Multiple single datasets	Not web-request specific, no cross-dataset test
Jagat et al. (2024) [8]	Variational LSTM autoencoder + deviation network	HTTP weblogs	Single dataset	No content-based branch, single dataset
Kamal and Mashaly (2025) [9]	Hybrid deep learning IDS	Network traffic	Single dataset	Single dataset, not web-log specific
Shiravi et al. (2012) [10]	Benchmark dataset design	Network traffic	Dataset study	Dataset paper, no detection model
Cantone et al. (2024) [19]	Multiple classical ML models	Network traffic	Cross-dataset	Not applied to web-log or hybrid deep models

Table 1 pulls this together in one place: what each study did, what kind of data it used, and where it falls short compared to what this paper does. Reading down the last column, the same few gaps keep repeating: most methods are checked on only one dataset, most do not combine a content view with a behaviour view, and none test whether a threshold set on one dataset still works on another. This paper is built to close those gaps together, rather than one at a time.

4. Materials and Methods

4.1. Research Design Overview

This study follows an experimental design. We build and train the models directly on real log data, then test them and report what we find. The setup is semi-supervised: both models are trained only on requests known to be normal, so no attack labels are needed to build the models themselves. Attack labels are only used afterwards, to check how well each trained model tells normal and attack traffic apart. The full process has five steps: preparing the data, building request-level and session-level features, training the two branch models, combining their scores through fusion, and setting a decision threshold. Each step is described below.

4.2. Datasets

We use eight datasets in total, listed in Table 2. One of them, the CSIC 2010 HTTP dataset, is a public dataset built by the Information Security Institute of CSIC (the Spanish National Research Council), for testing web attack detection on an e-commerce application [20]. It has 61,065 requests in total: 36,000 normal and 25,065 attack requests. It is the only dataset in this study that is not self-generated.

The rest of the datasets were generated for this project's own testbed, using normal browsing traffic mixed with known attack tools (such as sqlmap and nikto) run against a small test web application, following an approach also used elsewhere in the intrusion-detection literature for building test data [10]. Three of these self-generated logs are used as target datasets for the cross-dataset test described in Section 4.7: Access Eval Mix 2000, Access Eval Small 500, and Nginx JSON Eval 800, which is the only one of the three stored as structured JSON lines rather than a plain Apache-style text log. The remaining four datasets support the branch ablation study, the classical-baseline comparison, and the training of the session model: Access Attacks 200, Access Mixed 500, Access Small Benign, and Access Super Long Session 2000, which is entirely normal traffic from a single client sending a long run of requests, used only to help train the session model.

An important detail, revisited in Section 5, is that most of the self-generated logs are attack-heavy rather than normal-heavy. For example, 81% of Access Eval Mix 2000 is attack traffic. This matters because a threshold set directly on one of these datasets is not really a "normal traffic" threshold, since most of what it sees is not normal.

Table 2. Datasets used in this study.

Dataset	Format	Total	Normal	Attack	Role
CSIC 2010	Public, e-commerce app	61,065	36,000	25,065	Source domain; session training
Access Eval Mix 2000	Apache Combined	2,000	380	1,620	Target dataset
Access Eval Small 500	Apache Combined	500	107	393	Target dataset
Nginx JSON Eval 800	Structured JSON	800	174	626	Target dataset
Access Attacks 200	Apache Combined	200	36	164	Ablation, baselines
Access Mixed 500	Apache Combined	500	205	295	Ablation, baselines
Access Small Benign	Apache Combined	120	54	66	Ablation, baselines
Access Super Long Session 2000	Apache Combined	2,000	2,000	0	Session training only

Table 2 lays out all eight datasets side by side: their format, their size, how many normal and attack requests each one has, and what role each plays in the experiments that follow. The Normal and Attack columns show why CSIC 2010 was chosen as the source dataset for the cross-dataset transfer test (Section 4.7): it is the only dataset here with a large, genuinely normal-dominated portion of traffic (36,000 of 61,065 requests), which is what a threshold calibration needs. By contrast, the three target datasets (Access Eval Mix 2000, Access Eval Small 500, Nginx JSON Eval 800) are mostly attack traffic, which is exactly why a threshold set directly on them, as discussed above, is not a fair "normal traffic" threshold.

4.3. Data Preprocessing and Feature Extraction

Each log line is first parsed into a common set of fields: client IP address, timestamp, request method, path, response status, response size, and, where available, the user agent string. Three log line formats are supported: Apache Combined (which includes referrer and user agent), Apache Common (without those two fields), and structured JSON lines, so that Nginx JSON Eval 800 can be parsed with the same pipeline as the Apache-style logs.

From each parsed line, we build one feature vector with 11 numbers, listed in Table 3. This is a light, structural view of a request: it looks at the shape of the request and the server's response to it, not at the request body or query values directly. The same 11 numbers are used for both branches; the only difference is that the session branch groups these vectors into short sequences (Section 4.5), while the content branch scores one vector at a time.

Before training, feature values are scaled with a standard scaler (zero mean, unit variance), fitted separately for the content model and the session model, using only the data each model is trained on.

Table 3. The 11 features built from each parsed request.

Feature	Meaning
Method code	HTTP method mapped to a small integer (e.g., GET = 1, POST = 2)
Path length	Number of characters in the request path
Is API	1 if the path starts with /api/, else 0
Has query	1 if the path has a query string, else 0
Has file extension	1 if the path looks like a file (has a dot before any query string), else 0
Status code	Raw HTTP response status code
Status 2xx / 3xx / 4xx / 5xx	Four yes/no flags for which hundred-range the status code falls in
Log response size	Natural log of the response size in bytes

Table 3 lists all 11 numbers built from a single request. Most are simple structural counts or flags: the method, the path length, whether the path looks like an API call or a file, the status code and which hundred-range it falls in, plus one number built from the response size. None of them look inside the request body or the query values themselves, which is why, as noted above, this feature set is built to catch strange-looking requests and sequences, not necessarily a strange value hidden inside an otherwise normal-looking request.

4.4. Content-Based Autoencoder

The content branch is a small feed-forward autoencoder. The encoder has four layers that shrink the 11 input numbers down to a 4-number summary ($11 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$), each followed by a ReLU activation. The decoder mirrors this, growing the summary back up to 11 numbers ($4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 11$), with no activation on the last layer, since its output needs to match the original values directly.

The model is trained only on requests known to be normal, using the mean squared error between the input and its rebuilt version as the loss, optimised with Adam (learning rate 0.001). At test time, the same mean squared error is used as the anomaly score for each request: a normal-looking request should be rebuilt closely, so a high error suggests the request does not look like anything the model saw as normal.

4.5. Session-Based Variational LSTM Autoencoder

The session branch models short sequences of requests from the same client, rather than single requests on their own. A client is identified by the pair (IP address, user agent).

In the original thesis code, session windows were built by sliding a fixed-size window over the whole log in time order, without checking which client sent each request. A single window could therefore mix requests from several different clients. We treat this as a real bug, not a design choice, and fix it in this study: windows are now built only from the requests belonging to one client at a time, so a window of length 20 always holds 20 requests from a single real client, never a mix.

The model is a variational autoencoder built from two LSTM networks. An LSTM encoder reads a window of requests and produces a mean and a variance for a small latent space (size 8). A latent value is drawn from this distribution only during training; at test time, the mean is used directly, so scoring is repeatable rather than random. A second LSTM decoder then rebuilds the whole window from this one latent value, repeated at every step. The training loss adds the mean squared reconstruction error to a KL-divergence term that keeps the latent space close to a standard normal distribution, with the KL term weighted at 0.001. As with the content branch, training uses only normal traffic and the Adam optimiser (learning rate 0.001).

Building real per-client windows needs enough requests from the same client, so the session model is trained on normal traffic pooled from two sources: the normal-labelled requests in CSIC 2010 (36,000 requests, almost all from the same two clients) and the fully normal Access Super Long Session 2000 log (2,000 requests from a single client). Together these give 38,000 requests, which build into 7,594 real per-client session windows of length 20, using a stride of 5 requests between windows.

At test time, scoring also uses a real rolling buffer, one client at a time: the first 19 requests from a client fill the buffer, and a session score is only produced once 20 requests from that same client have arrived. This matches how windows are built during training. We call this the corrected session model for the rest of this paper, and compare it directly against the original, uncorrected version in Section 5.

4.6. Fusion and Anomaly Scoring

Once both branches produce a score for a request, the two scores are combined into one fused score by a weighted average:

$$s_{\text{fused}} = \frac{w_c \cdot s_c + w_s \cdot s_s}{w_c + w_s} \quad (1)$$

where s_c and s_s are the content and session scores and w_c, w_s are their weights. The content branch always scores every request; the session branch only scores a request once its client's buffer is full, so early requests from a new client are scored on content alone. If only one branch has a score for a given request, the fused score is simply that branch's score. The default weights are $w_c = w_s = 0.5$; Section 5 reports what happens when this balance is changed.

4.7. Threshold Selection and Cross-Dataset Transfer

A request is flagged as anomalous when its score is above a threshold. Following the approach used in the original thesis work, thresholds are set as a percentile of the score distribution, with the 95th percentile as the default, so only the highest-scoring 5% of requests are flagged.

Two ways of setting this threshold are compared in this paper. **Self-calibrated thresholding** sets the percentile threshold directly on the dataset being tested, using every request

in it, normal and attack alike. This is what the original thesis work did. **Cross-dataset threshold transfer** instead sets the threshold once, using only the normal-labelled requests from CSIC 2010, and then applies that one fixed threshold, unchanged, to the three target datasets, without ever looking at their own scores. This second approach is closer to how a real detector would be used: a threshold is set once, using traffic known to be normal, and then applied to new data it has never seen. Section 5 compares the two directly.

4.8. Evaluation Metrics

Because attack traffic is a small share of the total in most anomaly detection settings, and a large share in some of the self-generated datasets used here, accuracy on its own is not a useful measure. We report the area under the ROC curve (AUC) and the area under the precision-recall curve (PR-AUC), which do not depend on a single threshold, together with threshold-dependent measures: precision, recall, F1 score, and false positive rate (FPR), plus the full confusion matrix.

4.9. Implementation and Reproducibility

All models are implemented in Python, using PyTorch for the autoencoder and the variational LSTM autoencoder, and scikit-learn for scaling, classical baselines, and metric calculations. Both models are trained with a fixed random seed, so repeated training runs give the same result. Trained models, scalars, and score outputs for every dataset are saved to disk, so results can be reproduced without retraining.

5. Results

5.1. Effect of the Session Construction Fix

Table 4 compares the original (uncorrected) session model against the corrected model described in Section 4.5, on the two datasets that have any real per-client session structure to score: CSIC 2010, where almost all requests come from the same two clients, and Access Super Long Session 2000, a single client sending 2,000 requests in a row with no attack traffic at all, so no threshold-based score is meaningful there. Figure 1 shows the CSIC 2010 AUC and F1 values from this table side by side.

On CSIC 2010, the fix makes a large difference. The session branch's AUC rises from 0.63 with the original model to 0.97 with the corrected model, and its F1 score rises from 0.14 to 0.22, with precision reaching 1.00.

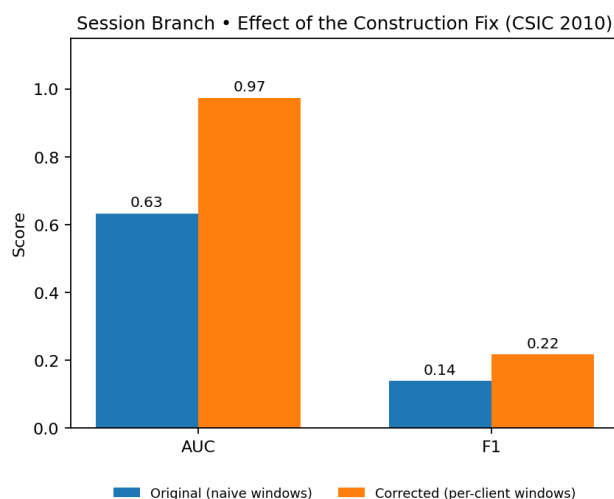
On the other six datasets (Access Eval Mix 2000, Access Eval Small 500, Nginx JSON Eval 800, Access Attacks 200, Access Mixed 500, Access Small Benign), the corrected model produces no session score at all, because, as Table 2 already suggests, almost every request in these logs comes from a different client. A true per-client model needs at least 20 requests from the same client before it can score anything, and these logs never reach that. This is not a weakness of the fix; it is a fact about the data. The original, uncorrected model scored every single line in these logs, but it did so by mixing unrelated clients into the same window, so that full coverage was not a real strength.

Table 4. Session branch, before and after the session construction fix.

Dataset	Model	Requests Scored	AUC	F1	Precision
CSIC 2010	Original	61,065	0.63	0.14	0.64
CSIC 2010	Corrected	61,065	0.97	0.22	1.00
Access Eval Mix 2000, and 5 others	Original	all lines	0.55–0.67	0.09–0.17	0.78–1.00
Access Eval Mix 2000, and 5 others	Corrected	0	—	—	—

Note: “—” means no score could be computed at all, not a score of zero. The corrected model needs at least 20 requests from the same client before it produces a score, and none of these six datasets has any client that sends that many requests, so the corrected model has nothing to score there.

Table 4 puts the two main facts about the session-branch fix in one place: on CSIC 2010, correcting the session windows raises AUC from 0.63 to 0.97 and F1 from 0.14 to 0.22, with precision reaching 1.00; and on every other dataset, the corrected model does not just score worse, it produces no score at all, for the reason given in the note below the table and in the paragraph above it.

**Figure 1.** Session branch AUC and F1 on CSIC 2010, before and after the session construction fix.

5.2. Branch Ablation

Table 5 reports AUC and F1 for the content, session, and fused branches on every dataset, using the corrected session model throughout. Figure 2 shows the same AUC values as a heatmap, making it easier to see where each branch does well.

The content branch reaches a moderate AUC (roughly 0.60–0.70) on most datasets, with high or perfect precision but low recall (roughly 0.06–0.09), because the 95th-percentile threshold only flags the most extreme-looking requests. This matches the precision-first design already used in the original thesis work: fewer, more trustworthy alerts over broad coverage.

The session branch only produces a score on CSIC 2010, for the reason given above. On that dataset it clearly beats the content branch (AUC 0.97 vs. 0.70).

The fused branch, on CSIC 2010, sits between the two single branches (AUC 0.77), better than content alone but worse than session alone. On the six datasets without a session score, the fused branch is identical to the content branch, since there is nothing to fuse with. Access Super Long Session 2000 is entirely normal traffic, so its AUC cannot be computed at all (there is only one class), and its F1 is 0 for every branch.

Table 5. Branch ablation: AUC and F1 by dataset, using the corrected session model.

Dataset	Content AUC	Session AUC	Fused AUC	Content F1	Session F1	Fused F1
CSIC 2010	0.70	0.97	0.77	0.14	0.22	0.14
Access Eval Mix 2000	0.69	—	0.69	0.12	—	0.12
Access Eval Small 500	0.70	—	0.70	0.12	—	0.12
Nginx JSON Eval 800	0.66	—	0.66	0.11	—	0.11
Access Attacks 200	0.47	—	0.47	0.11	—	0.11
Access Mixed 500	0.60	—	0.60	0.16	—	0.16
Access Small Benign	0.63	—	0.63	0.17	—	0.17
Access Super Long Session 2000	n/a	n/a	n/a	0.00	0.00	0.00

Table 5 is the main result of this section: it lines up the content, session, and fused branches on every dataset, using the corrected session model throughout. It shows the same pattern described above in numbers: the session column only has values for CSIC 2010, the fused branch matches the content branch exactly wherever no session score exists, and CSIC 2010 is the one row where the session branch clearly leads both other branches.

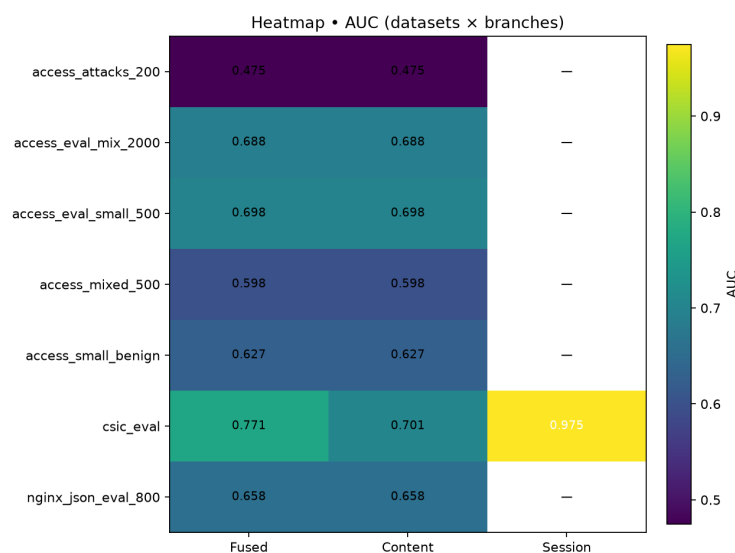
**Figure 2.** AUC by dataset and branch (columns left to right: fused, content, session).

Figure 2 shows the same numbers as Table 5, laid out as a colour grid with one row per dataset and one column for each branch, where darker cells mean a higher AUC. The session column is empty for every dataset except CSIC 2010, since that is the only dataset where the session branch produces a score at all. The one clearly dark cell in the whole grid is CSIC 2010's session column; the rest of the grid sits in a narrower, more medium range. This makes it easy to see, at a glance, that the session branch's strong result is one bright spot rather than a pattern repeated across datasets.

5.3. Fusion Weight Sensitivity

Because a session score only exists for CSIC 2010, this is the only dataset where changing the fusion weight has any effect. Table 6 sweeps the weight given to each branch, from content-only ($w_c = 1$) to session-only ($w_s = 1$).

AUC and F1 both rise steadily as more weight is put on the session branch, and the best score on both measures is reached at full session weight, not at any blended point in between, as shown in Figure 3. This means that, for this dataset, mixing in the content branch never helps; it only pulls the fused score away from the stronger session branch.

This goes against the usual assumption that fusing two branches is always better than picking the stronger one, and it is worth stating plainly rather than glossing over. On the other seven datasets, sweeping the weight has no effect, since only a content score exists there.

Table 6. Fusion weight sweep on CSIC 2010.

w_c	w_s	AUC	F1
1.0	0.0	0.70	0.14
0.7	0.3	0.75	0.14
0.5	0.5	0.77	0.14
0.3	0.7	0.78	0.15
0.0	1.0	0.97	0.22

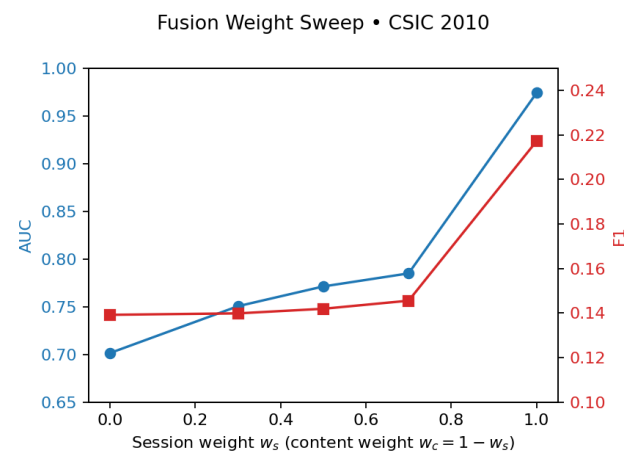


Figure 3. AUC and F1 on CSIC 2010 as the fusion weight shifts from content-only ($w_s = 0$) to session-only ($w_s = 1$).

Figure 3 shows both the AUC line and the F1 line climbing together as more weight moves toward the session branch, with no dip or bump anywhere in between. If mixing the two branches ever helped more than using the stronger branch alone, we would expect to see a peak somewhere in the middle of the plot. Instead, both lines reach their highest point only at the session-only end. This is the clearest way to see that, for this dataset, blending in the content branch only pulls the score down; it never lifts it above what the session branch reaches on its own.

5.4. Cross-Dataset Threshold Transfer

Table 7 compares the two ways of setting the decision threshold described in Section 4.7: self-calibrated (tuned separately on each target dataset, using every row in it) and transferred (set once on CSIC 2010's normal traffic only, then applied unchanged). AUC does not change between the two, since AUC does not depend on a threshold; only precision, recall, F1, and false positive rate change.

The result is the opposite of what we expected going in. Using the transferred threshold raises F1 on every target dataset, not by a small amount, but by roughly five to six times for the fused branch, and by roughly four to five times for the content branch alone. For example, on Access Eval Mix 2000, the fused branch's F1 rises from 0.12 (self-calibrated) to 0.67 (transferred). The same pattern holds on Access Eval Small 500 (0.12 to 0.67) and Nginx JSON Eval 800 (0.11 to 0.67), and at a slightly smaller multiplier for the content branch alone on all three datasets.

The reason is the point already raised in Section 4.2: these three target datasets are attack-heavy, not normal-heavy (81%, 79%, and 78% attack traffic, respectively). A threshold set at the 95th percentile of a dataset that is mostly attack traffic ends up far too high, since it is really the 95th percentile of a mostly-attack distribution, not a normal-traffic distribution, so it only catches the most extreme 5% of an already unusual mix. The frozen CSIC threshold, by contrast, is set on genuinely normal traffic, so it sits at a more sensible point and catches far more real attacks, at the cost of a higher false positive rate (up to about 0.30–0.36 on the fused branch, compared to 0.00–0.02 self-calibrated).

This is a useful, practical finding: on these datasets, what data a threshold is calibrated on matters more than whether it is calibrated on the same dataset being tested. A threshold set on real normal traffic, even from a completely different dataset, can work better than one set on the target dataset itself, if that target dataset does not actually represent normal conditions well. Figure 4 shows the self-calibrated and transferred F1 scores side by side for both branches.

Table 7. Self-calibrated vs. transferred threshold, on the three target datasets.

Target Dataset	Branch	AUC	Self F1	Self FPR	Transf. F1	Transf. Recall	Transf. FPR
Access Eval Mix 2000	Content	0.69	0.12	0.00	0.54	0.38	0.09
Access Eval Mix 2000	Fused	0.69	0.12	0.00	0.67	0.54	0.30
Access Eval Small 500	Content	0.70	0.12	0.00	0.51	0.35	0.04
Access Eval Small 500	Fused	0.70	0.12	0.00	0.67	0.53	0.24
Nginx JSON Eval 800	Content	0.66	0.11	0.02	0.54	0.38	0.09
Nginx JSON Eval 800	Fused	0.66	0.11	0.02	0.67	0.56	0.36

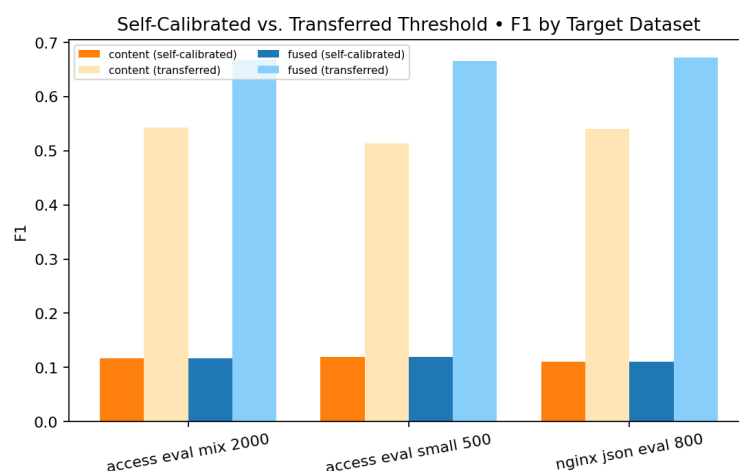


Figure 4. Self-calibrated vs. transferred-threshold F1, content and fused branches, by target dataset.

Figure 4 places the self-calibrated and transferred F1 scores side by side for each target dataset, for both the content and fused branches. In every pair of bars, the transferred-threshold bar is much taller than the self-calibrated bar next to it, across all three datasets and both branches. This makes the size of the improvement easy to see at a glance: it is not a small edge in one or two cases, it is a large, consistent jump everywhere this comparison is made.

5.5. Comparison with Classical Baselines

Table 8 compares our content and fused branches with three classical baselines: Random Forest, which is trained using attack labels, unlike our own models, and Isolation

Forest and One-Class SVM, which are trained only on normal traffic, the same setup as our own models.

Random Forest usually reaches the highest AUC of all methods (for example, 0.90 on Access Attacks 200 and 0.89 on CSIC 2010), which is expected, since it is the only method here allowed to see attack labels during training. Its F1 score is also high, but this comes with a much higher false positive rate than our fused branch in every case (for example, 0.55 on Access Eval Mix 2000, compared to 0.00 for our fused branch on the same dataset). This trade-off matters in a real security setting, where too many false alerts can make a system less useful even when its raw detection rate looks strong.

Isolation Forest and One-Class SVM, which do not use attack labels, generally score lower than Random Forest on AUC, but their comparison against our own branches is mixed. Our fused branch clearly beats both unsupervised baselines on four of the seven comparable datasets (CSIC 2010, Access Eval Mix 2000, Access Eval Small 500, and Nginx JSON Eval 800). On two more datasets (Access Mixed 500 and Access Small Benign), the result is split: our fused branch beats one of the two baselines but not the other. On the remaining dataset, Access Attacks 200, both unsupervised baselines clearly beat our fused branch, with Isolation Forest reaching an AUC of 0.86 against our 0.47, the largest gap seen anywhere in this comparison. The one dataset where our system does clearly best of all methods, including Random Forest, is CSIC 2010, where the session branch alone reaches an AUC of 0.97 (Table 5), higher than every method or baseline listed in Table 8, which does not include a session column since the session branch produces no score on the other datasets there. Figure 5 plots all five methods' AUC side by side across every dataset.

Table 8. Comparison with classical baselines.

Dataset	Content AUC	Fused AUC	RF AUC	RF FPR	IForest AUC	OCSVM AUC
CSIC 2010	0.70	0.77	0.89	0.30	0.71	0.44
Access Eval Mix 2000	0.69	0.69	0.78	0.55	0.66	0.62
Access Eval Small 500	0.70	0.70	0.75	0.58	0.66	0.66
Nginx JSON Eval 800	0.66	0.66	0.68	0.62	0.55	0.55
Access Attacks 200	0.47	0.47	0.90	0.20	0.86	0.61
Access Mixed 500	0.60	0.60	0.68	0.44	0.57	0.64
Access Small Benign	0.63	0.63	0.72	0.25	0.76	0.47

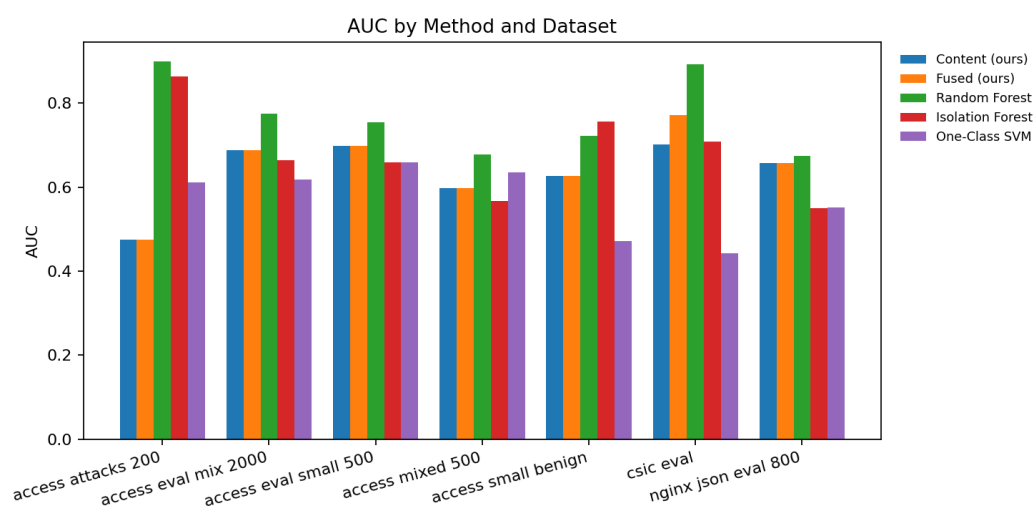


Figure 5. AUC by method and dataset: our content and fused branches against Random Forest, Isolation Forest, and One-Class SVM.

Figure 5 lines up all five methods' AUC scores side by side for every dataset, so the pattern in Table 8 can be seen directly. Random Forest's bar is usually the tallest, since it is the only method trained with attack labels. Our fused branch's bar is ahead of both Isolation Forest and One-Class SVM on four of the seven datasets, mixed on two more, and clearly behind both on Access Attacks 200, where Isolation Forest's bar is noticeably taller than ours.

5.6. Inference Speed

We also measured how fast the system runs, using the Access Eval Mix 2000 log. Scoring one request at a time, as a live system would, the content branch takes about 0.14 ms per request on average, and the session branch about 0.08 ms, giving an overall throughput of about 4,420 requests per second. Scoring in batches instead, the same log runs at about 630,000 lines per second, roughly 140 times faster than one-at-a-time scoring. Both numbers are fast enough for near-real-time use on ordinary hardware, with batch scoring far ahead whenever requests can be grouped before scoring.

5.7. Summary of Results

Taken together, these results show three things. First, the session branch is powerful, but only where the traffic actually contains real client sessions; the fix in Section 4.5 changes its usefulness from close to a coin flip to strong detection on the one dataset, CSIC 2010, where it can genuinely operate. Second, fusion is not automatically better than picking the stronger branch: on CSIC 2010, weighting fully toward the session branch beats every blended fusion weight tested. Third, how a threshold is set matters as much as which model produces the score: a threshold frozen from real normal traffic, even from a different dataset, outperformed a threshold tuned on the target dataset itself, because several of the target datasets used here are attack-heavy rather than normal-heavy.

6. Discussion

The results in Section 5 support the case, made in Section 1, that content-based and behaviour-based detection are complementary rather than interchangeable, but they also complicate the usual assumption that combining the two is always the better choice. Prior hybrid, fusion-based work has generally reported that combining detection signals improves over any single-model detector [9], and the session branch here does exactly what that line of work would predict on CSIC 2010, comfortably outperforming the content branch alone once its session windows are built correctly. What the fusion-weight sweep in Table 6 shows, however, is that blending the two branches on that same dataset is strictly worse than using the session branch on its own: every mixed weight tested sits between the two single-branch scores, and the best result is at full session weight. This is not a contradiction of the hybrid-detection literature so much as a boundary condition on it – fusion helps when the two branches make comparably reliable but different mistakes, and it hurts when one branch is reliably much stronger than the other on the traffic being scored, since averaging then just pulls the fused score toward the weaker signal. A fixed 50/50 weighting, of the kind used as a sensible default in the absence of other information, is therefore not a safe assumption to carry into a new dataset without checking it.

The session-branch results also speak directly to the caveat raised about behaviour-based detection in Section 3: a sequence model can only model a sequence if one actually exists in the data. The correction described in Section 4.5 is a straightforward bug fix – windows should never mix requests from different clients – but its effect size (AUC rising from 0.63 to 0.97 on CSIC 2010) shows how badly a session model can be misled by session windows that are not really sessions at all. The other side of that same fix is that six of

the eight datasets used here have essentially no client with 20 or more requests to build a window from, so the corrected model produces no score on them whatsoever. This is a genuine limitation of session-based detection as a technique, not an artefact of this particular implementation, and it means that a deployment decision about whether to run a session branch at all should be based on measuring the client request-count distribution of the target traffic first, rather than assumed to be generally useful the way the content branch is.

The cross-dataset threshold-transfer result in Table 7 is the most counter-intuitive finding in this study, and it is worth being explicit about why it happens rather than treating it only as a number. A percentile threshold is only a meaningful proxy for "unusually different from normal traffic" if the distribution it is computed on is itself mostly normal traffic. Three of the target datasets used here are between 78% and 81% attack traffic (Section 4.2), so a threshold set at their 95th percentile is really being set against a distribution made up mostly of attack scores, not mostly normal scores. This pushes the threshold far higher than it would be on real normal traffic, since it has to sit above a large share of already-high attack scores rather than just the tail end of normal behaviour, which is why recall and F1 are so low in the self-calibrated columns of Table 7. This is consistent with the broader finding in the cross-dataset generalisation literature that a fixed evaluation protocol can hide large differences in how well a threshold transfers [19], but the direction of the effect here is the opposite of the usual warning: instead of a model looking better than it really is because it was tuned on its own test data, the self-calibrated approach makes the framework look far *worse* than it is, because the calibration data was not the kind of traffic a percentile threshold assumes. The practical implication is that a percentile threshold should be calibrated on traffic that is known, or at least believed with reasonable confidence, to be predominantly normal, even if that traffic comes from a different source than the traffic being monitored, rather than calibrated automatically on whatever traffic happens to be available at deployment time.

Against the classical baselines, the pattern in Table 8 reinforces a point made in Section 4.7 about what this framework is optimised for. Random Forest reaches the highest AUC in most cases, which is unsurprising given it is the only method trained with attack labels, but it does so with a much higher false positive rate than the fused branch on every dataset compared: on five of the seven datasets the fused branch has no false positives at all, while Random Forest's false positive rate ranges from 0.20 to 0.62. In a real deployment, false positives carry a direct operational cost – alerts that must be triaged by a human – so a detector with a lower AUC but a substantially lower false-positive rate is not automatically the worse choice, particularly for a semi-supervised setting where attack labels are assumed unavailable in the first place. The unsupervised baselines (Isolation Forest, One-Class SVM) are a fairer comparison, since they share the same no-attack-label constraint as this framework. Against these, the fused branch clearly beats both on four of the seven comparable datasets, gives a mixed result on two more (beating one baseline but not the other), and loses clearly on one, Access Attacks 200, where Isolation Forest is the stronger method. Combined with the inference-speed results in Section 5, which show the framework comfortably meets near-real-time throughput requirements, this suggests the precision-first design carried over from the original thesis work (Section 4) is a reasonable operating point for a system meant to generate trustworthy rather than exhaustive alerts.

This study has several limitations that should temper how far its findings are generalised. First, the session branch's strong result is demonstrated on only one dataset, CSIC 2010, because it is the only dataset in this study with enough per-client requests to build real session windows; the claim that a corrected session model helps is well supported where it applies, but it has not been shown to help on genuinely session-rich traffic of a

different shape or scale. Second, several of the self-generated datasets are small (as few as 120 to 500 requests), which limits the statistical precision of the metrics computed on them and means single-point AUC and F1 differences between datasets should be read as indicative rather than tightly bounded. Third, the 11-feature representation (Table 3) is deliberately structural and does not inspect request bodies or query values directly, so it is unlikely to catch attacks whose payload is anomalous but whose structural shape (method, path length, status code, response size) looks unremarkable; this is a deliberate scope choice rather than an oversight, but it bounds what the content branch can be expected to catch. Fourth, the self-generated attack traffic was produced with a small number of well-known tools (sqlmap and nikto), so it may not represent the full diversity of real-world attack techniques or evasion strategies. Fifth, all results are reported from a single training and evaluation run per configuration, without repeated trials, cross-validation, or confidence intervals, so, as with the fixed random seed noted in Section 4, the reported numbers should be read as reproducible rather than as statistically confirmed against run-to-run variance. Sixth, all evaluation here is offline, on fixed logs; the inference-speed measurements in Section 5 indicate the framework’s computational profile is compatible with real-time use, but the framework has not actually been run against a live traffic stream, so behaviour under real deployment conditions such as concept drift or partially arriving sessions remains untested.

These limitations point to concrete future work. Testing the corrected session model on additional datasets with genuine, larger-scale multi-request sessions would show whether the CSIC 2010 result generalises or is specific to that dataset’s traffic pattern. Extending the feature representation to include lightweight payload or query-string signals, without giving up the efficiency of the current structural features, could let the content branch catch attacks that do not alter a request’s structural shape. Because the threshold-transfer result depends entirely on how “normal enough to calibrate on” is judged, an adaptive or likelihood-based thresholding scheme that estimates its own confidence in the calibration data, rather than assuming a fixed percentile is always appropriate, is a natural next step. Finally, validating the framework against live or replayed traffic, and evaluating it against a wider and more current set of attack tools, would test the real-time and generalisation claims made here under conditions closer to genuine deployment.

7. Conclusions

This paper presented a hybrid anomaly detection framework for web server logs that fuses a content-based autoencoder, scoring individual HTTP requests on their structural features, with a session-based variational LSTM autoencoder, scoring genuine per-client request sequences. The framework was evaluated across eight datasets spanning three log formats – self-generated Apache-style logs, a structured Nginx JSON log, and the public CSIC 2010 HTTP dataset – under both a standard branch ablation and a strict cross-dataset threshold-transfer protocol, rather than the single-dataset evaluations that dominate much of the existing literature on this problem (Section 3).

Three findings stand out. First, correcting a genuine session-construction fault, which previously let session windows mix requests from different clients, raised the session branch’s AUC on CSIC 2010 from 0.63 to 0.97, but the same corrected model produced no score at all on the six datasets that lack real multi-request client sessions to build windows from. A session-based branch is therefore only as useful as the session structure actually present in the traffic, and this should be checked before assuming one will help. Second, fusing the two branches is not automatically better than using the stronger one alone: on CSIC 2010, the only dataset where both branches produce a score, weighting fully toward the session branch beat every blended fusion weight tested, which goes against the usual

assumption that combining detection signals is always advantageous. Third, a decision threshold calibrated once on genuinely normal CSIC 2010 traffic and transferred unchanged to three attack-heavy target datasets raised fused F1 by roughly five to six times over a threshold tuned separately on each target dataset (for example, 0.12 to 0.67 on Access Eval Mix 2000), because those target datasets contain too much attack traffic for a self-calibrated percentile threshold to represent normal behaviour at all. Alongside these three findings, the framework achieved a substantially lower false-positive rate than a labelled Random Forest baseline, matched or exceeded unsupervised Isolation Forest and One-Class SVM baselines on most datasets, and processed requests fast enough for near-real-time use, at roughly 630,000 lines per second in batch mode.

Taken together, these results argue that in fusion-based web-log anomaly detection, how session windows are built and how detection thresholds are calibrated can matter as much as which model architecture is used, and that both should be evaluated explicitly rather than assumed to transfer safely from one dataset or deployment to another. The main limitations of this study – a single dataset with genuine session structure to demonstrate the session branch’s benefit on, a structural rather than payload-aware feature representation, and evaluation on offline logs rather than live traffic (Section 6) – point directly to future work: testing the corrected session model on additional session-rich datasets, extending the feature representation to lightweight payload signals, developing an adaptive thresholding scheme that can judge how normal its own calibration data is, and validating the framework against live or replayed traffic. Overall, this work shows that a carefully constructed, dataset-aware hybrid detector can generalise across genuinely different web log formats and traffic compositions, provided its session modelling and threshold calibration are matched to the structure of the data it is actually applied to, rather than treated as fixed design choices.

Author Contributions: A.R.: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Original Draft Preparation, Writing—Review and Editing, Visualisation. R.U.: Conceptualization, Validation, Writing—Review and Editing, Supervision, Project Administration. M.H.: Conceptualization, Validation, Writing—Review and Editing, Supervision. All authors have read and agreed to the published version of the manuscript.

Funding: This work received no external funding.

Institutional Review Board Statement: Not applicable. This research did not involve human participants or animals. The self-generated traffic logs were produced against a test application built for this study, and the CSIC 2010 dataset is a pre-existing, publicly available dataset.

Informed Consent Statement: Not applicable.

Data Availability Statement: The CSIC 2010 HTTP dataset is publicly available from the Information Security Institute of CSIC (Spanish National Research Council). The self-generated Apache- and Nginx-style log datasets, together with the trained models, scalars, and score outputs used to produce the results in Section 5, are available from the corresponding author upon reasonable request.

Acknowledgments: The University of the West of Scotland is thanked for providing the academic environment and computational facilities for conducting this research. The authors also thank their supervisors for their guidance and constructive feedback throughout this work.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

HTTP	Hypertext Transfer Protocol
IDS/IPS	Intrusion Detection/Prevention System
LSTM	Long Short-Term Memory
VAE	Variational Autoencoder
KL	Kullback–Leibler (divergence)
AE	Autoencoder
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
PR	Precision–Recall
F1	F1 Score (harmonic mean of precision and recall)
FPR	False Positive Rate
RF	Random Forest
SVM	Support Vector Machine
OCSVM	One-Class Support Vector Machine
JSON	JavaScript Object Notation
API	Application Programming Interface

References

- Scarfone, K.; Mell, P. *Guide to Intrusion Detection and Prevention Systems (IDPS)*; NIST Special Publication 800-94; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2007. <https://doi.org/10.6028/NIST.SP.800-94>.
- Chandola, V.; Banerjee, A.; Kumar, V. Anomaly detection: A survey. *ACM Comput. Surv.* **2009**, *41*, 15. <https://doi.org/10.1145/1541880.1541882>.
- Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Comput.* **1997**, *9*, 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Nedelkoski, S.; Bogatinovski, J.; Acker, A.; Cardoso, J.; Kao, O. Self-Attentive Classification-Based Anomaly Detection in Unstructured Logs. In Proceedings of the 2020 IEEE International Conference on Data Mining (ICDM), Sorrento, Italy, 17–20 November 2020; pp. 1196–1201. <https://arxiv.org/abs/2008.09340>.
- Du, M.; Li, F.; Zheng, G.; Srikumar, V. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), Dallas, TX, USA, 30 October–3 November 2017; pp. 1285–1298.
- Kingma, D.P.; Welling, M. Auto-Encoding Variational Bayes. In Proceedings of the 2nd International Conference on Learning Representations (ICLR), Banff, AB, Canada, 14–16 April 2014. <https://arxiv.org/abs/1312.6114>.
- An, J.; Cho, S. Variational Autoencoder based Anomaly Detection using Reconstruction Probability; SNU Data Mining Center Technical Report 2015-2; Seoul National University: Seoul, Republic of Korea, 2015.
- Jagat, R.R.; Sisodia, D.S.; Singh, P. Detecting Web Attacks From HTTP Weblogs Using Variational LSTM Autoencoder Deviation Network. *IEEE Trans. Serv. Comput.* **2024**, *17*, 2210–2222. <https://doi.org/10.1109/TSC.2024.3453748>.
- Kamal, H.; Mashaly, M. Enhanced Hybrid Deep Learning Models-Based Anomaly Detection Method for Two-Stage Binary and Multi-Class Classification of Attacks in Intrusion Detection Systems. *Algorithms* **2025**, *18*, 69. <https://doi.org/10.3390/a18020069>.
- Shiravi, A.; Shiravi, H.; Tavallaee, M.; Ghorbani, A.A. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Comput. Secur.* **2012**, *31*, 357–374. <https://doi.org/10.1016/j.cose.2011.12.012>.
- Liu, F.T.; Ting, K.M.; Zhou, Z.-H. Isolation Forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM), Pisa, Italy, 15–19 December 2008; pp. 413–422. <https://doi.org/10.1109/ICDM.2008.17>.
- Chua, W.; Pajas, A.L.D.; Castro, C.S.; Panganiban, S.P.; Pasuquin, A.J.; Purganan, M.J.; Malupeng, R.; Pingad, D.J.; Orolfo, J.P.; Lua, H.H.; et al. Web Traffic Anomaly Detection Using Isolation Forest. *Informatics* **2024**, *11*, 83. <https://doi.org/10.3390/informatics11040083>.
- Schölkopf, B.; Platt, J.C.; Shawe-Taylor, J.; Smola, A.J.; Williamson, R.C. Estimating the Support of a High-Dimensional Distribution. *Neural Comput.* **2001**, *13*, 1443–1471. <https://doi.org/10.1162/089976601750264965>.
- Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. <https://doi.org/10.1023/A:1010933404324>.
- Pan, Y.; Sun, F.; Teng, Z.; White, J.; Schmidt, D.C.; Staples, J.; Krause, L. Detecting web attacks with end-to-end deep learning. *J. Internet Serv. Appl.* **2019**, *10*, 16. <https://doi.org/10.1186/s13174-019-0115-x>.
- Tiwari, K.; Bhatia, A.S.; Garg, N.; Arora, I.; Saini, P. Comparative Analysis of CNN and Transformers on Malicious Intent Detection in HTTP. In *The Future of Artificial Intelligence and Robotics*; Springer: Cham, Switzerland, 2024; pp. 438–453. https://doi.org/10.1007/978-3-031-60935-0_40.

17. Zhou, L.; Yau, W.-C.; Gan, Y.S.; Liong, S.-T. E-WebGuard: Enhanced neural architectures for precision web attack detection. *Comput. Secur.* **2025**, *148*, 104127. <https://doi.org/10.1016/j.cose.2024.104127>. 739
18. Guo, H.; Yuan, S.; Wu, X. LogBERT: Log Anomaly Detection via BERT. In Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, 18–22 July 2021; pp. 1–8. <https://doi.org/10.1109/IJCNN52387.2021.9534113>. 740
19. Cantone, M.; Marrocco, C.; Bria, A. Machine learning in network intrusion detection: A cross-dataset generalization study. *IEEE Access* **2024**, *12*, 144489–144508. <https://doi.org/10.1109/ACCESS.2024.3472907>. 741
20. Giménez, C.T.; Villegas, A.P.; Marañón, G.A. *HTTP Dataset CSIC 2010*; Information Security Institute of CSIC (Spanish Research National Council): Madrid, Spain, 2010. 742